NeuralGeoQA: Neural Question Answering for Geospatial Knowledge Graphs over Wikidata

MSc Thesis Progress Report
Department of Informatics and Telecommunications
National and Kapodistrian University of Athens

[Student Name]
Supervisor: Prof. Manolis Koubarakis

April 23, 2026

Contents

1 Motivation **2**

2 Datasets **2**

2.1 Wikidata SimpleQuestions 2

2.2 GeoQuestions1089 3

2.3 GeoQuestions201 3

3 System Architecture **4**

3.1 Walkthrough: "Which rivers are in Wales?" 5

3.2 Stage 1: Neural Question Understanding 7

3.3 Stage 2: Entity Linking 8

3.4 Stage 3: SPARQL Query Construction 8

3.5 Stage 4: Execution and Answer Formatting 8

4 Evaluation **8**

4.1 Question Type Classification 8

4.2 End-to-End Results on GeoQuestions201 9

4.3 Error Analysis 9

5 Key Findings **9**

6 Relation to GeoQA2 **10**

7 Remaining Work and Future Directions **10**

1 Motivation

Geospatial question answering allows users to ask natural language questions about geographic entities and receive structured answers from a knowledge graph. For example, given the question "Which rivers are in Wales?", a system should query a knowledge graph, retrieve all river entities located in Wales, and return their names.

The GeoQA2 system, developed in our research group, addresses this task using rule-based NLP over the YAGO2geo knowledge graph. While effective, this approach has two limitations: (a) the handcrafted rules require significant engineering effort and are brittle to new question patterns, and (b) the system is tied to YAGO2geo, which is no longer actively maintained.

Meanwhile, recent work in general Knowledge Graph Question Answering (KGQA) has shown that pretrained transformer models like BERT can be fine-tuned to understand questions and extract structured information from knowledge graphs, achieving strong results on benchmarks like SimpleQuestions.

This thesis investigates a natural question: *Can we replace the rule-based NLP components of geospatial QA with neural alternatives, and can we target Wikidata—the largest, actively maintained open knowledge graph—instead of YAGO2geo?*

2 Datasets

This work uses three datasets, each serving a different role in the pipeline.

2.1 Wikidata SimpleQuestions

Source. The `askplatypus/wikidata-simplequestions` dataset on GitHub, originally derived from the Freebase SimpleQuestions benchmark and mapped to Wikidata by Diefenbach et al.

Structure. Each example is a triple: (subject entity, Wikidata property, object entity) paired with a natural language question. For example:

```
Q276304   P136   Q11399   ‘‘what is the genre of on the floor’’
```

Filtering. The full dataset contains 43,637 examples. We use only the *answerable* (ANS) subset where the gold triple exists in the current Wikidata dump: 19,481 training, 2,821 validation, and 5,622 test examples covering 125 unique Wikidata properties.

Role. This dataset is used for **Phase 1**: pretraining the BERT encoder with span detection and relation classification heads. It teaches the model general question understanding over Wikidata.

Note on R-prefix relations. The dataset uses "R-prefix" relations to encode inverse properties. For example, R19 is the inverse of P19 (place of birth), meaning "born here." During inference, we convert R-prefix predictions to P-prefix and note the directionality, though most downstream templates hardcode the correct geospatial property and do not rely on the predicted relation.

2.2 GeoQuestions1089

Source. Published by our research group as part of the GeoQA2 system. Available on HuggingFace at `AI-team-UoA/GeoQuestions1089`.

Structure. 1,089 natural language geospatial questions with gold SPARQL queries targeting YAGO2geo. Questions cover UK and Ireland geography: administrative regions, rivers, mountains, landmarks, points of interest.

Auto-annotation. The original dataset contains only questions and SPARQL queries, without explicit labels for question type, entities, or relations. We developed an automatic annotation pipeline that parses the gold SPARQL queries to extract:

- **QType**: question category (12 fine-grained types, derived from the SPARQL structure)
- **Entity1, Entity2**: the geographic entities mentioned
- **Relation**: the Wikidata property or spatial operation
- **AnswerType**: the expected answer entity type (e.g., river, county)
- **Distance, Numeric Constraint**: extracted thresholds (e.g., "within 5km", "more than 1000m")

The auto-annotation pipeline achieves 95.2% coverage: Entity1 is resolved for 998/1,089 questions, Relation for 1,055/1,089 (34 marked as NEEDS_REVIEW), and AnswerType for 1,026/1,089. Two questions overlapping with GeoQuestions201 were removed, leaving 1,087 for training.

Question type distribution. The distribution is heavily imbalanced:

Table 1: QType distribution in GeoQuestions1089 (training set).

QType	Count	QType	Count
E_class_near	307	B_boolean	15
G_count	216	F_thematic_spatial	15
G_superlative	179	E_class_distance	7
B_spatial	156	C_class_spatial_rel	4
A_attribute	150	G_comparative	3
C_class_in	33	B_directional	2

Role. This dataset is used for **Phase 2**: fine-tuning the QType classification head. Only the QType label is used for training; the Entity, Relation, and AnswerType columns serve as reference for the pipeline but are not used as training targets.

2.3 GeoQuestions201

Source. A subset of 201 questions from the GeoQuestions benchmark (Punjani et al., 2018), published by our research group as the evaluation set for GeoQA2.

Gold answer annotation. The original dataset contains only questions and gold SPARQL queries (targeting YAGO2geo). Since our system targets Wikidata, we cannot execute the gold SPARQL queries directly. Instead, we manually annotated gold answers for 196 out of 201 questions (5 marked as UNKNOWN or DATA_DEPENDENT). The annotation format varies by question type:

Table 2: Gold answer format by question type.

QType	Format	Example
B_boolean, B_spatial	Yes / No	"No"
G_count	Integer	"48"
A_attribute	Text value	"Argyll and Bute, Scotland"
G_superlative	Entity name	"River Tay"
C_class_in	Semicolon list	"Towy; Usk; Dee; Severn; Taff"
E_class_near	List or DATA_DEPENDENT	"DATA_DEPENDENT"

QType corrections. During annotation, we identified 4 questions where the auto-assigned QType was incorrect and corrected them:

- Q23, Q141: G_comparative → F_thematic_spatial (filtered lists, not comparisons)
- Q111: C_class_in → A_attribute (asks for area value, not entity list)
- Q200: B_spatial → G_superlative (asks for nearest, not existence)

Role. This dataset is used for **out-of-distribution evaluation**. The question type distribution differs significantly from GeoQuestions1089 (e.g., B_boolean is 15% of the test but only 1% of training).

3 System Architecture

NeuralGeoQA is a four-stage pipeline. Each stage receives the output of the previous one and adds structured information. Figure 1 shows the overall architecture.

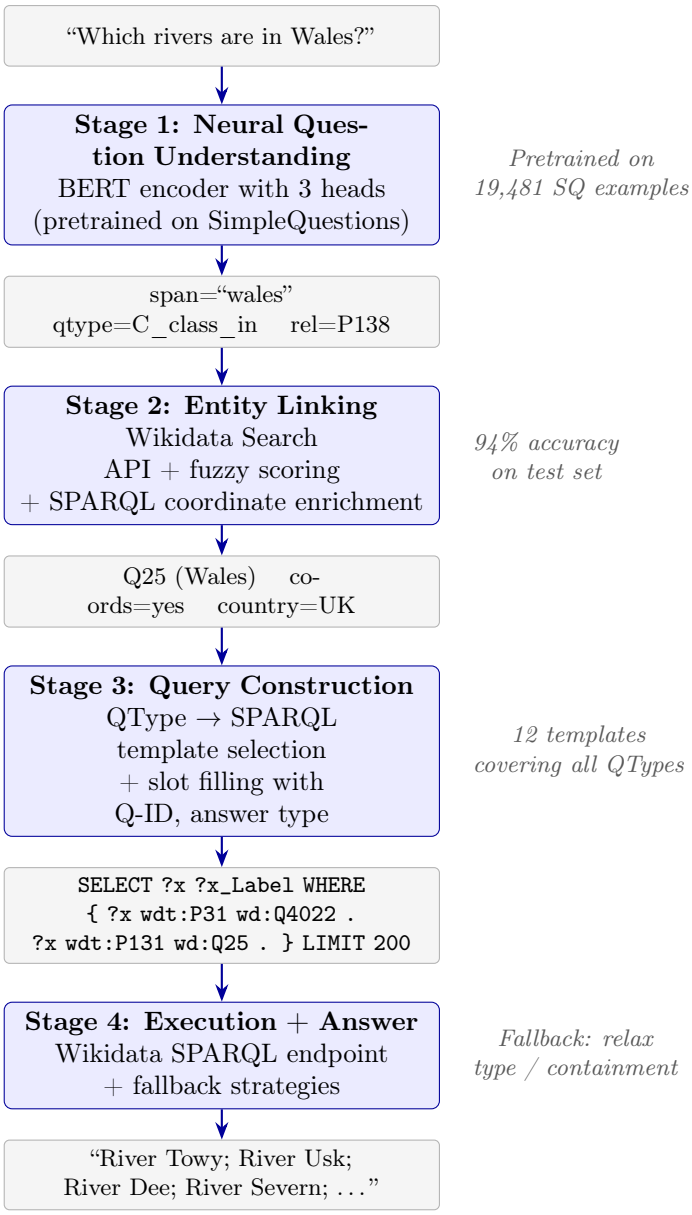

Figure 1: The NeuralGeoQA pipeline. Each stage adds structured information: the neural model extracts question intent, entity linking grounds it in Wikidata, template selection generates SPARQL, and execution returns the answer.

3.1 Walkthrough: "Which rivers are in Wales?"

To make the pipeline concrete, we trace a single question through all four stages.

Stage 1: Neural Question Understanding. BERT reads the question as a token sequence and runs three heads simultaneously. The *span head* predicts start/end token positions of the subject entity and points to "wales" — the thing we need to look up in Wikidata. The *QType head* classifies the question's structure as `C_class_in`, meaning "give me all entities of a certain class that are contained in a location." This tells Stage 3 which SPARQL template to use. The *relation head* predicts `P138` here, but for geospatial questions this prediction is largely ignored. The template for `C_class_in` hardcodes the correct Wikidata properties (`P31`, `P131`) regardless. The relation head is more useful for non-geospatial questions from SimpleQuestions.

Stage 2: Entity Linking. The span `"wales"` is sent to the Wikidata Search API. Multiple candidate Q-IDs come back. Fuzzy string scoring ranks them. A SPARQL enrichment query then fetches metadata (P31 type, P625 coordinates, P131 admin location, P17 country) for the top candidates in one batch call. The winner is **Q25**, the Wikidata item for Wales. The enrichment confirms it has coordinates and is in the United Kingdom, which boosts confidence and rules out disambiguation candidates such as a city named Wales in the United States.

Stage 3: Query Construction. QType `C_class_in` selects the "class contained in location" template. Two slots are filled:

```
SELECT ?x ?xLabel WHERE {
?x wdt:P31  wd:Q4022 .   % slot 1: answer type (river)
?x wdt:P131 wd:Q25   .   % slot 2: linked entity (Wales)
} LIMIT 200
```

Slot 1 uses `wdt:P31` (instance of) with `Q4022` (Wikidata's item for *river*). The system infers "river" from the question's noun: the answer-type detector reads the word "rivers" and maps it to Q4022. Slot 2 uses `wdt:P131` (located in administrative territorial entity) with `Q25` (Wales). Together the two triple patterns mean: *find all ?x that are rivers AND are located in Wales.* The `?xLabel` variable is resolved by the Wikidata label service, so the endpoint returns "River Towy" instead of "Q621344". `LIMIT 200` caps the result set to avoid timeout on large containment queries.

Stage 4: Execution and Answer. The query hits the Wikidata SPARQL endpoint. If it returns results, they are joined into a semicolon-separated list. If it returns nothing, fallback strategies kick in:

- *Relax type:* drop the `P31 Q4022` triple, returning anything contained in Wales (catches rivers typed as `Q55659167` "watercourse" instead of `Q4022` "river").
- *Relax containment:* try `P17` (country) instead of `P131`, or traverse containment transitively.

The final answer: `"River Towy; River Usk; River Dee; River Severn; ..."`

Why this works end-to-end. Each stage narrows the problem:

Stage	Input	Output	Narrowing
1	Raw question string	Intent + entity span	What kind of query?
2	Span "wales"	Grounded ID: Q25	Which KG entity?
3	QType + Q25	Executable SPARQL	What to ask the KG?
4	SPARQL query	Natural language answer	What is the answer?

The critical insight is that `C_class_in` questions always follow the same SPARQL skeleton: two triple patterns, one for type, one for containment. BERT's job is just to recognize that skeleton from the question wording. The hardcoded template then does the heavy lifting.

3.2 Stage 1: Neural Question Understanding

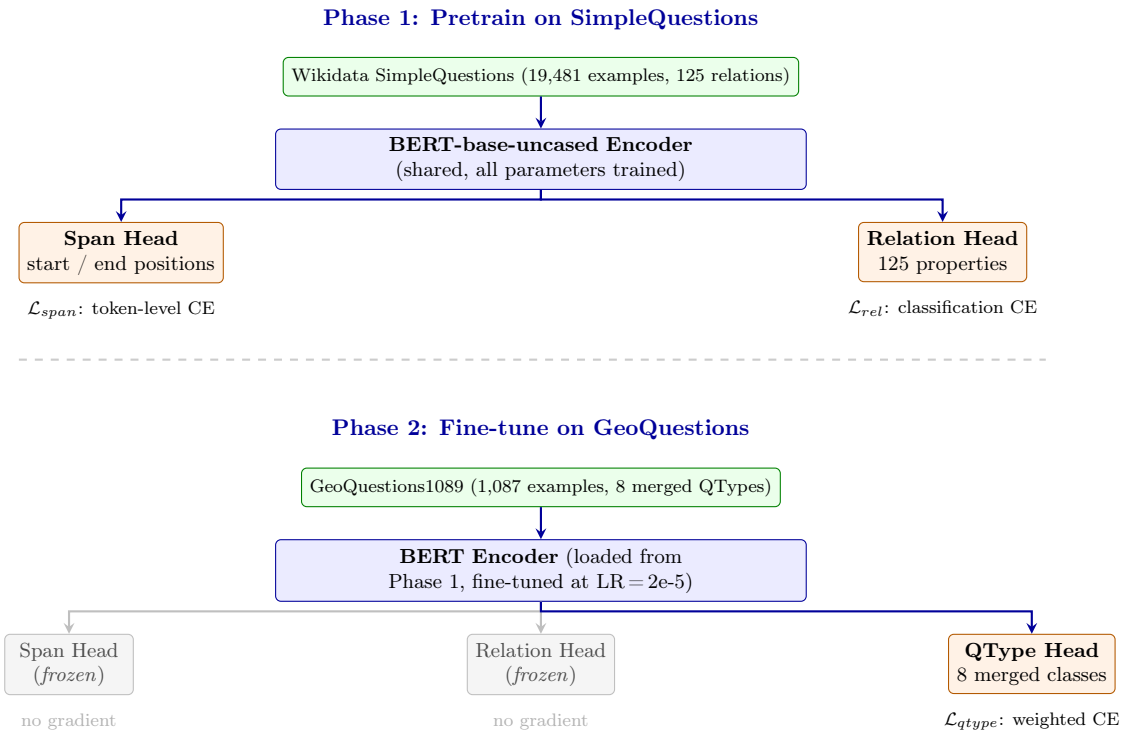

Figure 2: Two-phase training of the three-head BERT model. In Phase 1, the span and relation heads are trained jointly on SimpleQuestions. In Phase 2, these heads are frozen (gray) and a new QType head (orange) is added and trained on GeoQuestions with weighted cross-entropy loss. The encoder is fine-tuned with a smaller learning rate to preserve pretrained representations.

The core of the system is a BERT-base-uncased encoder with three classification heads, trained in two phases:

Phase 1: Pretraining on SimpleQuestions. The span head and relation head are trained jointly on 19,481 Wikidata SimpleQuestions examples. The span head identifies the subject entity by predicting start and end token positions. The relation head classifies the Wikidata property from the `[CLS]` representation.

Table 3: SimpleQuestions pretraining results (answerable subset).

	Our Two-Head	Our Three-Head	Original Paper
Span Accuracy	79.4%	79.0%	95.2%
Span F1 (avg)	89.7%	89.1%	97.5%
Relation Accuracy	95.3%	95.0%	83.5%
Object Accuracy	—	22.5%	—
Training examples	19,481		75,722
Unique relations	125		∼1,800

Phase 2: Fine-tuning on GeoQuestions. A QType classification head is added to the pretrained encoder. The span and relation heads are frozen; only the QType head and encoder are trained on 1,087 GeoQuestions with differential learning rates (encoder: 2e-5, QType head: 3e-4).

To address severe class imbalance (Table 1), two strategies are applied:

1. **Class merging:** 12 classes → 8 classes. Categories with fewer than 10 examples are merged into their nearest parent (e.g., C_class_spatial_rel → C_class, B_directional → B_spatial).
2. **Weighted cross-entropy:** Each class is weighted inversely to its frequency.

At inference, merged predictions are refined back into 12 types using keyword rules (e.g., B_spatial + "north of" → B_directional). Additionally, rule-based overrides correct two systematic BERT failure cases where E_class_near and B_spatial predictions are confused with other classes.

3.3 Stage 2: Entity Linking

The predicted entity span is linked to a Wikidata Q-ID through candidate retrieval (Wikidata search API with variant generation), fuzzy string scoring, SPARQL enrichment of top candidates (fetching P31 type, P625 coordinates, P131 admin location, P17 country in a single batch query), and secondary entity extraction via regex for multi-entity questions. Distance and numeric constraints are also extracted. All results are cached to disk for efficiency.

3.4 Stage 3: SPARQL Query Construction

Based on the classified QType, a SPARQL template is selected and filled with the linked entity Q-ID, detected answer type, and extracted constraints. Twelve template types cover all GeoQuestions categories, using properties such as P131 (containment), P47 (borders), P177 (crosses), P625 (coordinates via wikibase:around), and ordering properties P2044/P2043/P2046 for superlatives.

3.5 Stage 4: Execution and Answer Formatting

Queries are executed against the Wikidata SPARQL endpoint with rate limiting (1.5s between calls), retry logic on HTTP 429 errors, and fallback strategies (relax containment transitivity, drop type constraints).

4 Evaluation

4.1 Question Type Classification

Table 4: QType classification accuracy.

Setting	**Merged (8)**	**Fine-grained (12)**
In-distribution (164 held-out)	67.1%	58.5%
Out-of-distribution (GeoQuestions201)	75.6%	75.6%

Structured types (G_count, G_superlative, A_attribute) achieve 91–100% in both settings. Spatial types (E_class_near: 2% in-distribution) remain the hardest, with proximity confused with containment in 59% of cases.

4.2 End-to-End Results on GeoQuestions201

Table 5: Pipeline performance on GeoQuestions201 (199 evaluated).

Metric	Value
Question Type Accuracy	75.6%
Entity Linking Success	94.0%
Pipeline Execution Success	77.6%
Answer Accuracy (strict)	16.1%
Answer Accuracy (lenient)	21.6%

4.3 Error Analysis

Table 6: Root cause distribution of errors.

Error cause	% of errors
Wikidata lacks required triples	38%
No template matched question pattern	18%
Count over-inclusive (type hierarchy)	9%
Entity linking failed	8%
QType mismatch → wrong template	8%
Wrong entity returned	8%
Boolean answer inverted	7%
List with zero recall	4%

Template diagnostics confirm that SPARQL structure is generally correct. Boolean, containment, and crossing queries return valid results when Wikidata has the data. The primary failure is inconsistent or missing data: English counties use 4+ different type identifiers, and many entities lack P47 (border) triples entirely.

5 Key Findings

1. **Transfer learning works for question understanding.** 75.6% QType accuracy and 94% entity linking with only 1,087 fine-tuning examples.

2. **The bottleneck is query generation, not NLU.** 38% of errors are missing Wikidata triples.

3. **Wikidata's type system is inconsistent for geographic QA.** Unlike YAGO2geo, Wikidata lacks standardized geographic typing.

4. **Spatial question disambiguation is the hard open problem.** Containment vs proximity vs topology: 2–70% accuracy depending on complexity.

5. **First neural GeoKBQA system on Wikidata.** All prior systems target YAGO2geo.

6 Relation to GeoQA2

Table 7: Comparison of GeoQA2 and NeuralGeoQA.

	GeoQA2	**NeuralGeoQA (ours)**
Question understanding	Rule-based NLP	Fine-tuned BERT
Entity recognition	Stanford CoreNLP	BERT span + Wikidata API
Target KG	YAGO2geo	Wikidata
Query generation	Pattern → templates	QType → templates
Training data	None (rule-based)	19,481 + 1,087 examples

Neural components replace rule-based NLP effectively, but the query generation challenge persists as a KG data quality problem.

7 Remaining Work and Future Directions

Remaining. GPT-4o baseline comparison, detailed error categorization, full thesis document, GitHub repository.

Future. Learned text-to-SPARQL generation (T5/BART), Wikidata geospatial normalization layer, spatial-aware embeddings, hybrid KG + LLM retrieval.